

```

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
import torchvision.models as models
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np
import copy

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

from google.colab import files
uploaded = files.upload()

<IPython.core.display.HTML object>

Saving style.jpg to style.jpg
Saving content.jpg to content.jpg

def load_image(path, max_size=400):
    image = Image.open(path).convert("RGB")

    transform = transforms.Compose([
        transforms.Resize(max_size),
        transforms.ToTensor()
    ])

    image = transform(image).unsqueeze(0)
    return image.to(device)

content = load_image("content.jpg")
style = load_image("style.jpg")

def imshow(tensor, title=None):
    image = tensor.clone().detach().cpu().squeeze(0)
    image = transforms.ToPILImage()(image)
    plt.imshow(image)
    if title:
        plt.title(title)
    plt.axis("off")
    plt.show()

imshow(content, "Content Image")
imshow(style, "Style Image")

```

Content Image



Style Image



```
vgg = models.vgg19(pretrained=True).features.to(device).eval()
```

```

for param in vgg.parameters():
    param.requires_grad = False

/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
since 0.13 and may be removed in the future, please use 'weights'
instead.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for
'weights' are deprecated since 0.13 and may be removed in the future.
The current behavior is equivalent to passing
`weights=VGG19_Weights.IMAGENET1K_V1`. You can also use
`weights=VGG19_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)

Downloading: "https://download.pytorch.org/models/vgg19-dcbb9e9d.pth"
to /root/.cache/torch/hub/checkpoints/vgg19-dcbb9e9d.pth

100%|██████████| 548M/548M [00:03<00:00, 177MB/s]

content_layer = '21' # conv4_2
style_layers = ['0', '5', '10', '19', '28']

def get_features(image, model):
    features = {}
    x = image

    for name, layer in model._modules.items():
        x = layer(x)
        if name == content_layer:
            features['content'] = x
        if name in style_layers:
            features[name] = x

    return features

content_features = get_features(content, vgg)
style_features = get_features(style, vgg)

def gram_matrix(tensor):
    b, c, h, w = tensor.size()
    tensor = tensor.view(c, h*w)
    gram = torch.mm(tensor, tensor.t())
    return gram

style_grams = {layer: gram_matrix(style_features[layer]) for layer in
style_features}

def content_loss(target, content):
    return torch.mean((target - content)**2)

```

```

def style_loss(target, style_gram):
    target_gram = gram_matrix(target)
    return torch.mean((target_gram - style_gram)**2)

target = content.clone().requires_grad_(True).to(device)

optimizer = optim.Adam([target], lr=0.003)

steps = 1500
content_weight = 1e4
style_weight = 50

for i in range(steps):

    target_features = get_features(target, vgg)

    # Content loss
    c_loss = content_loss(target_features['content'],
                          content_features['content'])

    # Style loss
    s_loss = 0
    for layer in style_layers:
        target_feature = target_features[layer]
        style_gram = style_grams[layer]
        s_loss += style_loss(target_feature, style_gram)

    total_loss = content_weight * c_loss + style_weight * s_loss

    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()
    with torch.no_grad():
        target.clamp_(0, 1)

    if i % 400 == 0:
        print(f"Step {i}, Total Loss: {total_loss.item()}")

Step 0, Total Loss: 950126848.0
Step 400, Total Loss: 944883264.0
Step 800, Total Loss: 933698944.0
Step 1200, Total Loss: 923605248.0

imshow(target, "Stylized Image")

```

Stylized Image

